SeatRush: Swagger UI Test Guide

Everything to click, type and expect on the /docs page

Open `https://<your-service>.onrender.com/docs` in your browser. No PowerShell is needed for anything in this guide. Work top to bottom and **stop at the first test that does not match**.

Part 0: How Swagger UI works (read once)

1. Click an endpoint row (for example **GET /health**) to expand it.
2. Click **Try it out** (top right of the expanded box). The fields become editable.
3. Fill the fields exactly as the guide says, then click the blue **Execute** button.
4. Scroll to **Server response**. Compare the **Code** (200, 201, 409...) and the **Response body** with the green box.
5. Values such as `hold_id`, `order_id` and `ticket` are not remembered by Swagger. **Copy them from the response body** and paste them into the next test.

Three things that trip people up:
- **Request bodies come pre-filled with placeholder text** such as `"string"`. Always delete it and paste the body from this guide. The JSON must be valid (straight double quotes, no trailing comma), otherwise you get **422**.
- **Header fields** (`x-user-id`, `idempotency-key`, `x-queue-ticket`) appear in the Parameters list with a red 'required' mark. Type your value in the box; Swagger will not run the request while a required box is empty.
- **The first Execute can take up to a minute** on the free tier (the service sleeps when idle) and may show 'Failed to fetch'. Wait and click Execute again.

Fake users: any text works as `x-user-id`. This guide uses **alice**, **bob**, **carol** and **dave** so you can act as different buyers. Seats: names like `S000-R00-00`. If you ran earlier tests, the first seats may already be sold, so **always use seat ids from your own seats list** (Test 5).

Part 1: Read-only checks (is everything connected?)

Test 1: Server is up

Confirms the web service is running.

Open: **GET** `/health` then click **Try it out**

EXPECTED RESPONSE (Code + Response body)

```
Code 200
{"status":"ok"}
```

Test 2: Database is connected and seeded (Neon)

Lists the two demo events created on first boot.

Open: **GET** `/events` then click **Try it out**

EXPECTED RESPONSE (Code + Response body)

```
Code 200
[{"id":1,"name":"Open Sale Concert","venue":"City Hall","starts_at":"...","waiting_room":false},
 {"id":2,"name":"Mega Tour Flash Sale","venue":"Stadium","starts_at":"...","waiting_room":true}]
```

500 here means the Neon connection strings in Render are wrong.

Test 3: Single event, and a missing event

Event 1 has no waiting room; event 2 does.

Open: **GET** `/events/{event_id}` then click **Try it out**

3a. A real event

Field in Swagger	Kind	Type this
`event_id`	path	1

EXPECTED RESPONSE (Code + Response body)

```
Code 200
{"id":1,"name":"Open Sale Concert",...,"waiting_room":false}
```

3b. An event that does not exist

Field in Swagger	Kind	Type this
`event_id`	path	999

EXPECTED RESPONSE (Code + Response body)

```
Code 404
{"detail":"event not found"}
```

Test 4: Seat counts

A fresh database has 2000 seats per event. Results are cached for 1 second.

Open: **GET** `/events/{event_id}/availability` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	1

EXPECTED RESPONSE (Code + Response body)

```
Code 200
{"available":2000,"held":0,"sold":0}
```

Numbers will differ if you already ran tests; available + held + sold should still add up to 2000.

Test 5: Browse free seats (and Redis check)

Lists free seats in order, 'keyset pagination'. The rate limiter and cache run on Redis, so success here means Redis works.

Open: **GET** `/events/{event_id}/seats` then click **Try it out**

5a. First page

Field in Swagger	Kind	Type this
`event_id`	path	1
`after`	query	leave empty
`limit`	query	3

EXPECTED RESPONSE (Code + Response body)

```
Code 200
{"seats":[{"seat_id":"S000-R00-00","price_cents":15000},{"seat_id":"S000-R00-01","price_cents":15000},
        {"seat_id":"S000-R00-02","price_cents":15000}],"next_after":"S000-R00-02"}
```

Write down the three seat ids. They are used in the next tests.

5b. Next page

Field in Swagger	Kind	Type this
`event_id`	path	1
`after`	query	S000-R00-02 (the next_after value)
`limit`	query	3

EXPECTED RESPONSE (Code + Response body)

```
Code 200
Three different seats starting at S000-R00-03.
```

Part 2: Buy a ticket on event 1 (open sale, no waiting room)

Test 6: Reserve a seat for alice

Holds the seat for 2 minutes so nobody else can buy it.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	1
`x-user-id`	header	alice

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": ["S000-R00-00"]}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 201
{"hold_id":"3f6c1d52-....","event_id":1,"seat_ids":["S000-R00-00"],
 "amount_cents":15000,"expires_at":"2026-...T...+00:00"}
```

Copy the hold_id. You need it in Tests 9, 10, 11 and 12. Use the first seat id from your Test 5a list if S000-R00-00 is gone.

Test 7: Nobody else can take it (no overselling)

The central guarantee of the project: a second buyer is refused immediately.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	1
`x-user-id`	header	bob

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": ["S000-R00-00"]}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 409
{"detail":"one or more seats are unavailable"}
```

If this returns 201 instead, that would be a double booking. Check that Test 6 succeeded first and that you used the same seat id.

Test 8: Bad requests are rejected

Try each of these four requests in turn. 8a to 8c use event_id 1 and x-user-id alice.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

8a. Empty list

Field in Swagger	Kind	Type this
`event_id`	path	1
`x-user-id`	header	alice

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": []}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 422
{"detail":[{"type":"too_short", ... "seat_ids" ...}]}
```

8b. More than 4 seats

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": ["S001-R00-00","S001-R00-01","S001-R00-02","S001-R00-03","S001-R00-04"]}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 422
{"detail":[{"type":"too_long", ...}]}
```

8c. A seat that does not exist

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": ["NOPE-1"]}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 409
{"detail":"one or more seats are unavailable"}
```

8d. An event that does not exist (event_id 999)

Field in Swagger	Kind	Type this
`event_id`	path	999

REQUEST BODY: delete the default text and paste this

```
{"seat_ids": ["S000-R00-05"]}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 404
{"detail":"event not found"}
```

Test 9: Nobody can pay for someone else's hold

Bob tries to check out alice's hold.

Open: **POST** `/orders` then click **Try it out**

Field in Swagger	Kind	Type this
`x-user-id`	header	bob
`idempotency-key`	header	bob-1

REQUEST BODY: delete the default text and paste this

```
{"hold_id": "<paste alice's hold_id from Test 6>"}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 404
{"detail":"hold not found"}
```

Test 10: Alice buys her seat (checkout saga)

Creates an order, charges the simulated payment provider, confirms the seat.

Open: **POST** `/orders` then click **Try it out**

Field in Swagger	Kind	Type this
`x-user-id`	header	alice
`idempotency-key`	header	order-1

REQUEST BODY: delete the default text and paste this

```
{"hold_id": "<paste alice's hold_id from Test 6>"}
```

EXPECTED RESPONSE (Code + Response body)

```
About 90% of the time:
Code 201
{"order_id":"9a41....","status":"CONFIRMED","event_id":1,"seat_ids":["S000-R00-00"],
 "amount_cents":15000,"payment_ref":"pay_1a2b3c...","failure_reason":null}

About 10% of the time (simulated card decline, this is correct):
Code 201
{... "status":"FAILED", ... "failure_reason":"payment_declined"}
```

Copy the order_id. On FAILED the seat is released: repeat Test 6 (a new hold) and Test 10 with a new key such as order-2. A 410 'hold expired' means more than 2 minutes passed since Test 6.

Test 11: Retrying is safe (idempotency)

Press Execute again with exactly the same fields and body as Test 10. No second charge happens.

Open: **POST** `/orders` then click **Try it out**

Field in Swagger	Kind	Type this
`x-user-id`	header	alice
`idempotency-key`	header	order-1 (the same key)

REQUEST BODY: delete the default text and paste this

```
{"hold_id": "<the same hold_id>"}
```

EXPECTED RESPONSE (Code + Response body)

```
Code 201
The same order_id and status as in Test 10.
```

Test 12: An idempotency key cannot be borrowed

Bob tries to reuse alice's key.

Open: **POST** `/orders` then click **Try it out**

Field in Swagger	Kind	Type this
`x-user-id`	header	bob
`idempotency-key`	header	order-1

REQUEST BODY: delete the default text and paste this
```
{"hold_id": "<alice's hold_id>"}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 409
{"detail":"idempotency key belongs to another user"}
```

Test 13: Read an order back

Look up the order by its id.

Open: **GET** `/orders/{order_id}` then click **Try it out**

13a. Your order

Field in Swagger	Kind	Type this
`order_id`	path	

EXPECTED RESPONSE (Code + Response body)
```
Code 200
Same order_id, status and seat_ids as Test 10.
```

13b. An id that does not exist

Field in Swagger	Kind	Type this
`order_id`	path	00000000-0000-0000-0000-000000000000

EXPECTED RESPONSE (Code + Response body)
```
Code 404
{"detail":"order not found"}
```

Test 14: Seat counts changed

Repeat Test 4 (wait one second first because of the 1-second cache).

Open: **GET** `/events/{event_id}/availability` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	1

EXPECTED RESPONSE (Code + Response body)
```
Code 200
{"available":1999,"held":0,"sold":1}
```

If Test 10 returned CONFIRMED you should see sold 1. Holds that are still within their 2 minutes show up under held.

Test 15: Release a hold you no longer want

Hold a second seat, give it back, and see that someone else can then take it.

Open: **DELETE** `/holds/{hold_id}` then click **Try it out**

15a. First make a new hold (use POST /events/{event_id}/holds as in Test 6)

Field in Swagger	Kind	Type this
`event_id`	path	1
`x-user-id`	header	alice

REQUEST BODY: delete the default text and paste this
```
{"seat_ids": ["S000-R00-01"]}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 201
Copy the new hold_id.
```

15b. Release it (DELETE /holds/{hold_id})

Field in Swagger	Kind	Type this
`hold_id`	path	
`x-user-id`	header	alice

EXPECTED RESPONSE (Code + Response body)
```
Code 200
{"released":true}
```

15c. Release it again

Field in Swagger	Kind	Type this
`hold_id`	path	
`x-user-id`	header	alice

EXPECTED RESPONSE (Code + Response body)
```
Code 404
{"detail":"no active hold"}
```

15d. Now bob takes that seat (POST holds, event_id 1, x-user-id bob)

REQUEST BODY: delete the default text and paste this
```
{"seat_ids": ["S000-R00-01"]}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 201
A hold for bob. The seat was free again.
```

Test 16: Optional: holds expire after 2 minutes

Takes about 2.5 minutes of waiting.

Open: **POST** `/orders` then click **Try it out**

16a. As carol, hold seat S000-R00-02 (POST holds, event_id 1, x-user-id carol). Copy the hold_id and wait 2 minutes 10 seconds.

EXPECTED RESPONSE (Code + Response body)
```
Code 201 for the hold.
```

16b. Then try to pay (POST /orders)

Field in Swagger	Kind	Type this
`x-user-id`	header	carol
`idempotency-key`	header	carol-1

REQUEST BODY: delete the default text and paste this
```
{"hold_id": "<carol's hold_id>"}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 410
{"detail":"hold expired"}
```

A background loop also frees expired seats every 5 seconds, so GET /events/1/seats will list S000-R00-02 again.

Part 3: The waiting room on event 2 (flash sale)

Event 2 only lets people hold seats after they have been through the queue. A background loop admits waiting users every second.

Test 17: No ticket, no entry

Carol tries to hold a seat on event 2 without queueing.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	2
`x-user-id`	header	carol

REQUEST BODY: delete the default text and paste this
```
{"seat_ids": ["S000-R00-00"]}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 403
{"detail":"join the waiting room and wait to be admitted"}
```

Test 18: Join the queue

Gets carol a ticket in the first-in-first-out line.

Open: **POST** `/events/{event_id}/queue/join` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	2
`x-user-id`	header	carol

EXPECTED RESPONSE (Code + Response body)
```
Code 200
{"ticket":"5be2a8c1d3f04a77:carol","admitted":false,"position":1,"queue_length":1}
```

Copy the whole ticket including the part after the colon. Executing again returns the same ticket (you cannot jump the queue by re-joining). If the service just woke up, admitted may already be true.

Test 19: Wait to be admitted

Click Execute every few seconds until admitted turns true (usually under 10 seconds).

Open: **GET** `/events/{event_id}/queue/status` then click **Try it out**

19a. Your ticket

Field in Swagger	Kind	Type this
`event_id`	path	2
`ticket`	query	

EXPECTED RESPONSE (Code + Response body)
```
Code 200
First: {"admitted":false,"position":1,"queue_length":1}
Then:  {"admitted":true,"position":0}
```

Never admitted? The background loops only run while the service is awake and when RUN_WORKER_IN_API is 1. Check the Render Logs tab.

19b. A made-up ticket

Field in Swagger	Kind	Type this
`event_id`	path	2
`ticket`	query	abc

EXPECTED RESPONSE (Code + Response body)
```
Code 200
{"admitted":false,"position":null}
```

Test 20: A ticket cannot be borrowed

Dave tries to use carol's ticket.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	2
`x-user-id`	header	dave
`x-queue-ticket`	header	

REQUEST BODY: delete the default text and paste this
```
{"seat_ids": ["S000-R00-00"]}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 403
{"detail":"join the waiting room and wait to be admitted"}
```

Test 21: Hold a seat once admitted

First list a free seat with GET /events/{event_id}/seats (event_id 2, limit 1). Then hold it as carol, with her ticket.

Open: **POST** `/events/{event_id}/holds` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	2
`x-user-id`	header	carol
`x-queue-ticket`	header	

REQUEST BODY: delete the default text and paste this
```
{"seat_ids": ["S000-R00-00"]}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 201
A hold object (hold_id, seat_ids, amount_cents, expires_at).
```

Copy the hold_id. If the ticket was admitted more than 10 minutes ago it expires and you must join again.

Test 22: Pay for the flash-sale seat

Same checkout as Test 10.

Open: **POST** `/orders` then click **Try it out**

Field in Swagger	Kind	Type this
`x-user-id`	header	carol
`idempotency-key`	header	carol-order-1

REQUEST BODY: delete the default text and paste this
```
{"hold_id": "<hold_id from Test 21>"}
```

EXPECTED RESPONSE (Code + Response body)
```
Code 201
"status":"CONFIRMED" (about 90%) or "FAILED" with "payment_declined" (about 10%).
```

Part 4: Reports and metrics

Test 23: The correctness report (the most important check)

The server inspects its own data. All three invariants must be true.

Open: **GET** `/admin/stats/{event_id}` then click **Try it out**

Field in Swagger	Kind	Type this
`event_id`	path	1 (then repeat with 2)

EXPECTED RESPONSE (Code + Response body)

```
Code 200
{"seats":{"available":1998,"held":1,"sold":1},"orders_by_status":{"CONFIRMED":1},
 "failed_reasons":{},"confirmed_orders":1,"confirmed_seats":1,"confirmed_cents":15000,
 "net_captured_cents":15000,"outbox_backlog":0,"notifications":1,
 "invariants":{"no_oversell_sold_eq_confirmed_seats":true,
               "no_seat_in_two_orders":true,"payments_reconcile":true}}
```

Counts depend on how much you tested. **All three invariants must be true.** Any false value is a real bug: copy the whole response and send it to me. outbox_backlog should be 0 and notifications should grow after each order (wait ~10 seconds).

Test 24: Prometheus metrics

Counters for this running instance. They reset when the service restarts.

Open: **GET** `/metrics` then click **Try it out**

EXPECTED RESPONSE (Code + Response body)

```
Code 200  (plain text)
...
tm_holds_total{result="ok"} 6.0
tm_holds_total{result="conflict"} 2.0
tm_holds_total{result="not_admitted"} 2.0
tm_orders_total{status="CONFIRMED"} 2.0
...
```

Look for the lines starting with tm_. The numbers are examples.

Part 5: Optional, confirm the data in Neon

In the Neon console open **SQL Editor** and run these. Wait about 10 seconds after your orders (the simulated email sender fails 15% of the time and retries).

RUN THIS (Neon SQL Editor)

```
select status, count(*) from seats where event_id = 1 group by status;
select status, failure_reason from orders order by created_at desc limit 5;
select count(*) as unpublished from outbox where published_at is null;
select kind, body from notifications order by id desc limit 5;
```

EXPECTED RESPONSE (Code + Response body)

```
seats:         SOLD 1 / AVAILABLE ~1998 / HELD (only while a hold is still inside its 2 minutes)
orders:        CONFIRMED (or FAILED payment_declined)
unpublished:   0
notifications: order.confirmed | order <id> order.confirmed
```

Part 6: Quick reference

What each status code means

Code	Meaning in SeatRush
200 / 201	OK / created (holds and orders return 201)
403	Waiting-room event: you were not admitted, or the ticket is not yours
404	Event, order, hold or active hold not found

Code	Meaning in SeatRush
409	Seat unavailable, or an idempotency key owned by another user
410	The hold has expired (more than 2 minutes)
422	Your JSON body is invalid: placeholder text left in, empty or too many seats
429	Rate limited (30 requests/second per user). Clicking by hand will not reach this
503	Server overloaded, or a deploy in progress. Retry after a few seconds
Failed to fetch	The free service is waking from sleep. Wait a minute and Execute again

Values to copy and paste

What	Value
Users (x-user-id)	alice, bob, carol, dave
Hold body	{"seat_ids": ["S000-R00-00"]}
Order body	{"hold_id": ""}
Idempotency keys	any text, new for each new order (order-1, order-2, carol-order-1 ...)
Waiting-room event	event_id 2 (event 1 has no waiting room)
If seats run out	Neon SQL Editor: TRUNCATE notifications, outbox, orders, holds, seats, events RESTART IDENTITY CASCADE; then restart the Render service

Note: the concurrent-load proof (hundreds of buyers at once) cannot be done by hand in Swagger. It is the load test described in RUNBOOK.pdf, which ends with RESULT: PASS - zero oversells, ledger reconciles.